SEGUNDO DEPARTAMENTAL- EJERCICIO 7- COLA DINÁMICA

En esta práctica desarrollamos un programa que permite implementar una cola dinámica, el objetivo es administrar una colección de elementos enteros mediante una estructura de datos que sigue el principio FIFO, es decir, el primer elemento que insertamos es el primero que eliminamos. Para construir nuestra cola utilizamos nodos enlazados dinámicamente mediante apuntadores, lo que nos permite agregar elementos sin establecer un tamaño máximo fijo, además, implementamos diferentes métodos para insertar, eliminar, mostrar y consultar elementos, así como para verificar si la cola se encuentra vacía y finalmente, incorporamos un menú interactivo que facilita el uso de las operaciones y un destructor que libera la memoria utilizada por los nodos.

Errores y Correcciones: En la versión inicial, al momento de ejecutar en repetidas ocasiones alguna opción del menú la pantalla se llenaba información lo que causaba confusión al momento de comprobar la operación solicitada. Por lo que agregamos una nueva opción al menú mediante la estructura switch: “Limpiar pantalla”.

Código Completo:

```
#include <iostream>
#include <stdlib.h>
using namespace std;

// Clase que representa cada nodo de la cola
class Nodo {
public:
    int dato;
    Nodo* siguiente;

    // Constructor del nodo
    Nodo(int valor) {
        dato = valor;
        siguiente = NULL;
    }
};

// Clase que maneja la cola dinamica
class Cola {
private:
    Nodo* frente;
    Nodo* final;

public:
    // Constructor de la cola
    Cola() {
        frente = NULL;
        final = NULL;
    }

    // Destructor: libera la memoria de los nodos
    ~Cola() {
        while (frente != NULL) {
            Nodo* auxiliar = frente;
            frente = frente->siguiente;
            delete auxiliar;
        }
        final = NULL;
```

```
    }

    // Insertar elemento (ENQUEUE)
    void insertar(int valor) {
        Nodo* nuevo = new Nodo(valor);

        if (frente == NULL) {
            frente = nuevo;
            final = nuevo;
        } else {
            final->siguiente = nuevo;
            final = nuevo;
        }

        cout << "Elemento insertado correctamente.\n";
    }

    // Eliminar elemento (DEQUEUE)
    void eliminar() {
        if (frente == NULL) {
            cout << "La cola esta vacia.\n";
        } else {
            Nodo* auxiliar = frente;

            cout << "Elemento eliminado: "
                 << frente->dato << endl;

            frente = frente->siguiente;

            if (frente == NULL) {
                final = NULL;
            }

            delete auxiliar;
        }
    }

    // Mostrar elementos de la cola
    void mostrar() {
        if (frente == NULL) {
            cout << "La cola esta vacia.\n";
        } else {
            Nodo* actual = frente;

            cout << "\nElementos de la cola:\n";

            while (actual != NULL) {
                cout << actual->dato << endl;
                actual = actual->siguiente;
            }
        }
    }

    // Consultar el primer elemento
    void consultarFrente() {
        if (frente == NULL) {
            cout << "La cola esta vacia.\n";
        } else {
            cout << "Elemento al frente: "
                 << frente->dato << endl;
        }
    }
```

```
    // Consultar el ultimo elemento
    void consultarFinal() {
        if (final == NULL) {
            cout << "La cola esta vacia.\n";
        } else {
            cout << "Elemento al final: "
                 << final->dato << endl;
        }
    }

    // Verificar si la cola esta vacia
    bool estaVacia() {
        return frente == NULL;
    }
};

int main() {
    Cola cola;
    int opcion, valor;

    do {
        cout << "\n=== COLA DINAMICA EN POO ===\n";
        cout << "1. Insertar elemento (ENQUEUE)\n";
        cout << "2. Eliminar elemento (DEQUEUE)\n";
        cout << "3. Mostrar cola\n";
        cout << "4. Consultar frente\n";
        cout << "5. Consultar final\n";
        cout << "6. Verificar si esta vacia\n";
        cout << "7. Limpiar pantalla\n";
        cout << "8. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "Ingresa un numero: ";
                cin >> valor;
                cola.insertar(valor);
                break;

            case 2:
                cola.eliminar();
                break;

            case 3:
                cola.mostrar();
                break;

            case 4:
                cola.consultarFrente();
                break;

            case 5:
                cola.consultarFinal();
                break;

            case 6:
                if (cola.estaVacia())
                    cout << "La cola esta vacia.\n";
                else
                    cout << "La cola contiene elementos.\n";
                break;
```

```
            case 7:
                system("cls");
                break;

            case 8:
                cout << "Programa finalizado.\n";
                break;

            default:
                cout << "Opcion no valida.\n";
        }

    } while (opcion != 8);

    return 0;
}
```

Explicación del Código:

1. Declaración de la clase Nodo (class Nodo)

Creamos la clase Nodo, cuya función es representar cada elemento que forma parte de nuestra cola dinámica, cada nodo almacena un dato y un apuntador que permite conectarlo con el siguiente nodo.

2. Declaración de la clase Cola (class Cola)

Creamos la clase Cola, que se encarga de administrar todos los elementos de la estructura dinámica, dentro de ella declaramos los apuntadores necesarios para controlar el inicio y el final de la cola.

3. Declaración de los apuntadores (frente y final)

- frente: apunta al primer elemento de la cola, que será el siguiente en eliminarse.
- final: apunta al último elemento de la cola, donde agregamos los nuevos elementos.

4. Constructor de la clase Cola (Cola())

- frente = NULL: indica que no existe un primer elemento.
- final = NULL: indica que no existe un último elemento.

5. Método para insertar elementos (insertar(int valor))

Creamos el método insertar(), cuya función es agregar nuevos elementos al final de nuestra cola, esta operación también se conoce como ENQUEUE.

6. Método para eliminar elementos (eliminar())

Creamos el método eliminar(), cuya función es retirar el primer elemento de nuestra cola, esta operación se conoce como DEQUEUE, antes de eliminar cualquier nodo, verificamos si la cola está vacía. Si frente == NULL, significa que no existen elementos para eliminar, con esta condición evitamos intentar acceder a un nodo inexistente.

7. Creación del apuntador auxiliar (Nodo* auxiliar = frente)

Si la cola contiene elementos, creamos un apuntador llamado auxiliar que guarda la dirección del primer nodo, esto nos permite conservar la dirección del nodo que vamos a eliminar, mientras actualizamos el apuntador frente.

8. Actualización del frente (frente = frente->siguiente)

Con esto, el segundo elemento de la cola se convierte en el nuevo primero y queda listo para ser eliminado en una siguiente operación.

9. Verificación de la cola después de eliminar (if (frente == NULL))

Comprobamos si frente quedó en NULL después de eliminar el nodo, esto significa que acabamos de eliminar el último elemento de nuestra cola, así mantenemos correctamente los dos apuntadores cuando la cola queda vacía.

10. Liberación de memoria del nodo eliminado (delete auxiliar)

Utilizamos delete auxiliar para liberar la memoria que ocupaba el nodo eliminado, es importante realizar esta operación porque los nodos fueron creados dinámicamente mediante new.

11. Método para mostrar los elementos (mostrar())

Creamos el método mostrar(), que permite visualizar todos los elementos almacenados en nuestra cola, primero verificamos si frente == NULL. Si se cumple, mostramos un mensaje indicando que la cola está vacía.

12. Recorrido de la cola mediante un apuntador auxiliar (Nodo* actual = frente)

Cuando existen elementos, declaramos un apuntador llamado actual y lo inicializamos con la dirección del primer nodo, utilizamos este apuntador para recorrer la estructura sin modificar los apuntadores principales frente y final.

13. Método para consultar el primer elemento (consultarFrente())

Creamos el método consultarFrente(), cuya función es mostrar el valor del primer elemento de nuestra cola sin eliminarlo. Primero verificamos si la cola está vacía. Si contiene elementos, esta operación nos permite conocer cuál es el siguiente elemento que se eliminaría.

14. Método para consultar el último elemento (consultarFinal())

Creamos el método consultarFinal(), que permite consultar el último elemento almacenado en nuestra cola sin modificar su contenido, verificamos si final == NULL. Si la cola contiene elementos, mostramos el valor, esto nos permite conocer cuál es el último elemento que insertamos.

15. Método para verificar si la cola está vacía (estaVacia())

Implementamos el método estaVacia(), que utiliza un valor de tipo bool para indicar si nuestra cola contiene elementos, si frente es igual a NULL, el método devuelve true; de lo contrario, devuelve false.

16. Función principal (int main())

Dentro de ella creamos un objeto llamado cola de la clase Cola. También declaramos dos variables enteras:

- opcion: almacena la opción seleccionada por el usuario.
- valor: almacena el número que deseamos insertar en nuestra cola.

Utilizamos un ciclo do-while para mostrar un menú interactivo que permite seleccionar las operaciones disponibles y utilizamos la estructura switch para ejecutar una operación dependiendo del número que ingrese el usuario.